

나만의 AI 비서 구축 최종 보고서

— 금/은 시세 데이터 분석 × FastAPI × Firestore × Gemini 연동 —

1. 미션 개요

본 미션은 시계열 데이터를 분석하고, 그 결과를 데이터베이스에 저장한 뒤, AI가 이 데이터를 기반으로 대화하는 "내 상황을 아는 AI 비서" 웹 서비스를 구축하는 것을 목표로 한다. 관심 있는 시계열 데이터로 금(GC=F)/은(SI=F) 선물 시세를 선정하였다.

항목	내용
미션명	AI Agent 개발: 나만의 AI 비서 구축
분석 대상 데이터	금/은 선물 시세 (yfinance, GC=F / SI=F, 1년치 502개 이상)
백엔드	FastAPI + Firebase Firestore + Google Gemini API
프론트엔드	HTML / CSS / JavaScript (바닐라, 프레임워크 미사용)
배포	백엔드: Render / 프론트엔드: Vercel
보너스	A(Function Calling + MCP Server), B(시각화/내보내기/다크모드) 모두 이행

2. 과제 목표 달성 체크리스트

#	학습자가 설명할 수 있어야 하는 것	달성 여부
1	시계열 데이터를 분석하고 요약 정보를 만들어 서비스에서 활용하는 흐름	달성
2	FastAPI 프로젝트를 라우터/서비스 등으로 분리해 구성한 기준	달성
3	Pydantic을 활용한 요청 데이터 검증을 적용한 이유와 방식	달성
4	Firestore에 데이터를 저장하고 CRUD로 다루는 방법	달성
5	데이터 요약을 시스템 프롬프트에 주입하는 방식(컨텍스트 주입)의 원리	달성
6	배포 환경에서 CORS/환경변수/키 관리가 왜 필요한지	달성

3. 기능 요구사항 이행 현황

#	요구 내용	구현 위치 / 상태
1	Python 3.10+ / venv / 필수 패키지 구성	backend/venv, requirements.txt
2	시계열 데이터 100개 이상 준비 및 요약 정보 생성	prepare_seed_data.py → seed_data.json (금/은 총 502개+)
3	FastAPI 앱 초기화, CORS, /docs 확인	app/main.py
4	Firestore 연동, 서비스계정 키 환경변수 관리	app/services/firestore_service.py, .env
5	데이터 API 5종 (CRUD 4 + summary)	app/routers/data.py
6	대화기록 API (저장/목록/상세/삭제)	app/routers/conversations.py
7	AI 챗봇 API (컨텍스트 주입)	app/routers/chat.py
8	백엔드 배포(Render), /docs 확인 가능	https://gn-mission3-2-ai-assistant.onrender.com/docs

#	요구 내용	구현 위치 / 상태
9	웹 프론트엔드(바닐라 JS): 채팅/데이터관리/대화기록/요약	frontend/index.html, script.js, style.css
10	프론트엔드 배포(Vercel), README 작성	https://gn-mission3-2-frontend.vercel.app, README.md

4. 결과물 1 - 백엔드 (FastAPI + Firestore + Gemini)

라우터/서비스/모델 3계층으로 분리하고, AI 프로바이더는 전략 패턴(Strategy Pattern)으로 추상화하여 Gemini와 OpenAI를 환경변수(AI_PROVIDER) 값만으로 런타임 전환할 수 있도록 설계하였다. 개발 단계에서는 무료 티어가 넉넉한 Gemini(gemini-3.5-flash-lite)를 사용하였다.

```

backend/app/
├── main.py                # FastAPI 앱 초기화, CORS, 라우터 등록
├── routers/
│   ├── data.py           # 데이터 CRUD + summary/statistics/sync-latest
│   ├── conversations.py  # 대화 저장/목록/상세/삭제
│   └── chat.py           # AI 챗봇(컨텍스트 주입) API
├── services/
│   ├── firestore_service.py
│   ├── data_service.py   # 요약/통계 계산 로직
│   └── ai/
│       ├── base.py       # AIProvider 공통 인터페이스
│       ├── gemini_provider.py
│       ├── openai_provider.py
│       ├── factory.py     # AI_PROVIDER 값에 따라 프로바이더 반환
│       └── tools.py       # Function Calling용 도구 정의(프로바이더 독립)
└── models/               # Pydantic 요청/응답 스키마

```

핵심 흐름 - AI 챗봇 API(컨텍스트 주입): 1) /api/data/summary 조회 2) 요약을 시스템 프롬프트에 삽입 3) Gemini API 호출 4) 대화 내용을 conversations에 자동 저장.

```

SYSTEM_PROMPT_TEMPLATE = """당신은 데이터 분석 비서입니다.

[사용자 데이터 요약]
- 데이터 기간: {period}
- 총 레코드: {count}개
- 주요 지표: {metrics}
- 최근 트렌드: {trend}
- 금/은 비율: {ratio}

위 데이터를 기반으로 맞춤형 답변을 제공하세요."""

```

실행 방법: backend 폴더에서 venv 활성화 후 `pip install -r requirements.txt → uvicorn app.main:app --reload`. `http://127.0.0.1:8000/docs` 에서 Swagger UI로 전체 API를 확인할 수 있다.

5. 결과물 2 - 프론트엔드 (바닐라 HTML/CSS/JS)

프레임워크를 사용하지 않는 요구사항에 맞춰, 시계 추이 그래프도 외부 차트 라이브러리 없이 Canvas API로 직접 구현하였다.

다크모드는 CSS 변수 기반으로 전환되며 localStorage에 저장되어 재방문 시에도 유지된다.

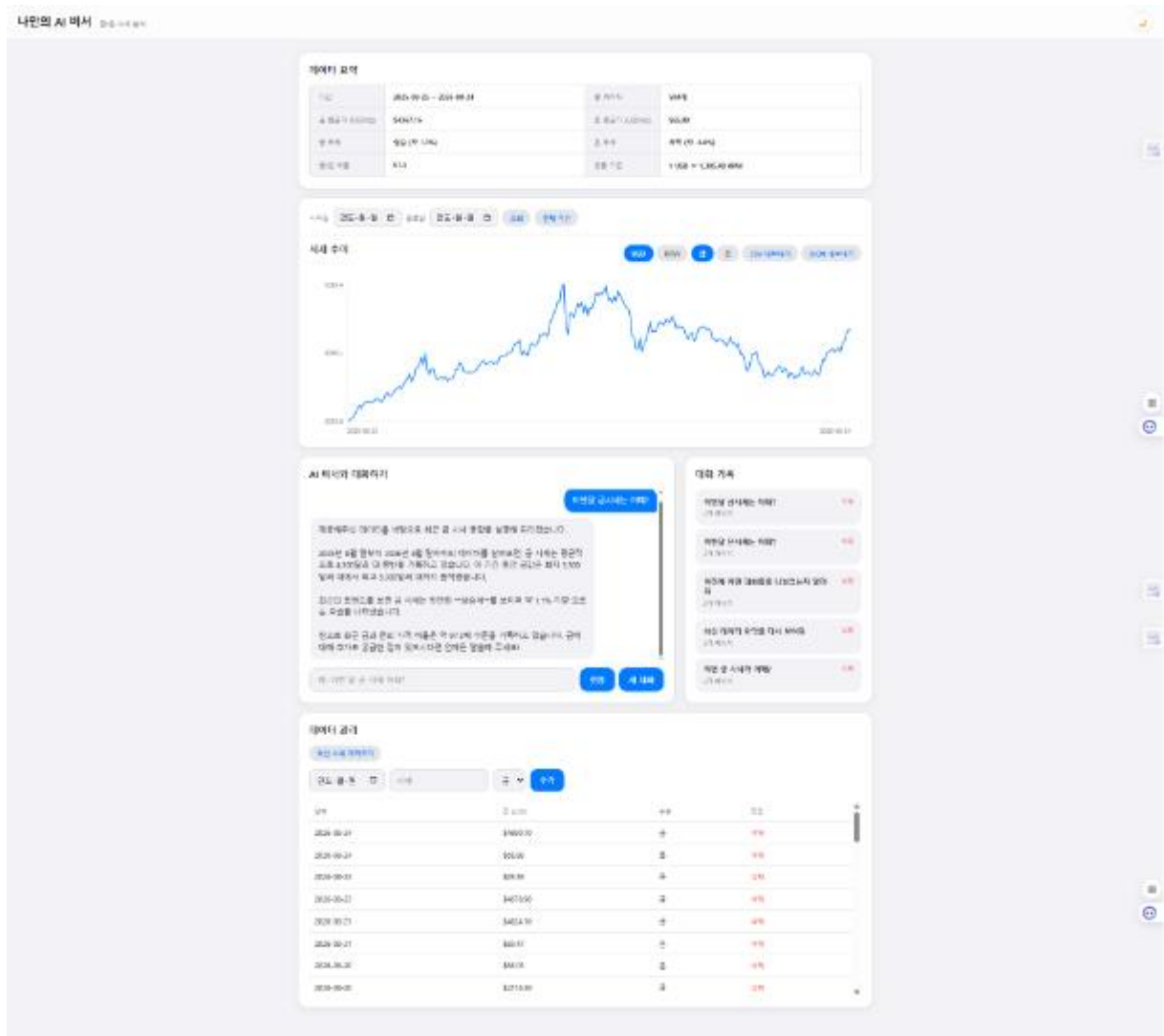


그림 1. 데스크톱 전체 화면 - 데이터 요약, 시세 추이 차트(기간 필터·통화 전환 포함), AI 채팅, 대화 기록, 데이터 관리가 한 화면에 통합되어 있다.

위 화면에서 사용자가 "이번달 금시세는 어때?"라고 질문하면, 시스템 프롬프트에 주입된 실제 Firestore 데이터(평균가, 최고/최저가, 추세)를 근거로 AI가 자연어로 답변한다. 상단 요약 카드에는 기간/총 데이터/평균가/추세/금은비율/환율 기준이 표로 정리되어 있으며, 중단에는 기간 필터·통화 (USD/KRW) 전환·CSV/JSON 내보내기 기능이 포함된 시세 추이 차트가 있다.

모바일 반응형 검증 - iPhone 실기기에서 다크모드 기준으로 채팅과 데이터 관리 화면을 확인하였다. 아래는 사용자가 "금을 구입하려는데 동향파악이 어떤가요?"라고 질문했을 때, AI가 실제 데이터를 근거로 답변하는 모바일 화면과, 데이터 관리 화면에서 "최신 시세 가져오기" 버튼(야후 파이낸스 연동)이 정상 노출되는 모바일 화면이다.

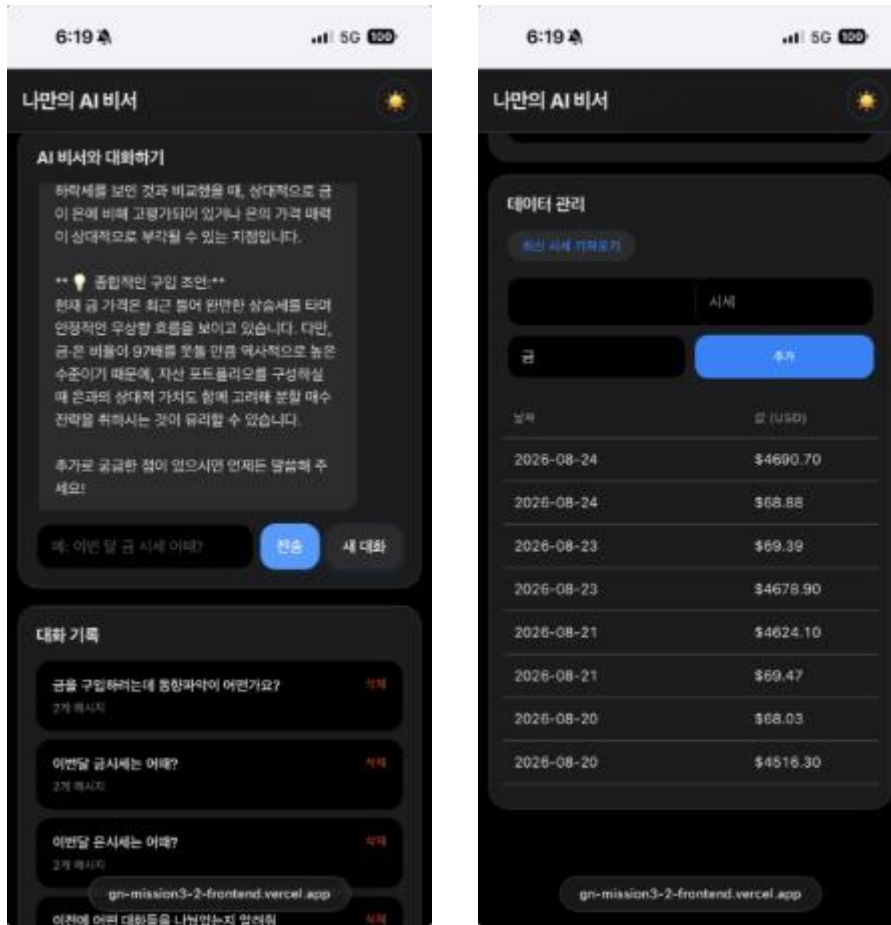


그림 2. 모바일(iPhone) AI 채팅 화면 — 데이터 근거 기반 답변

그림 3. 모바일(iPhone) 데이터 관리 화면 — 최신 시세 가져오기 버튼 포함

실행 방법: frontend/index.html을 VSCode Live Server로 열어야 한다(file:// 직접 열기는 CORS로 차단 됨). <http://127.0.0.1:5500/frontend/index.html>

6. 결과물 3 - 배포 환경

구분	URL / 값
백엔드(Render)	https://gn-mission3-2-ai-assistant.onrender.com
Swagger 문서	https://gn-mission3-2-ai-assistant.onrender.com/docs
프론트엔드(Vercel)	https://gn-mission3-2-frontend.vercel.app
GitHub 저장소	https://github.com/bzkhseo-source/3-2-ai-assistant

Render 환경변수(AI_PROVIDER, GEMINI_API_KEY, FIREBASE_SERVICE_ACCOUNT_JSON, ALLOWED_ORIGINS 등)에 로컬 .env와 동일한 값을 등록하여 배포 환경에서도 동일하게 동작하도록 구성하였다. Vercel의 배포 도메인을 Render 백엔드의 ALLOWED_ORIGINS에 추가 등록하여 CORS를 해결하였다.

나만의 AI 비서 - 금/은 시세 분석 서비스

Swagger UI

시세별 데이터(금/은 시세) 분석 서비스 API를 관리하는 Swagger UI

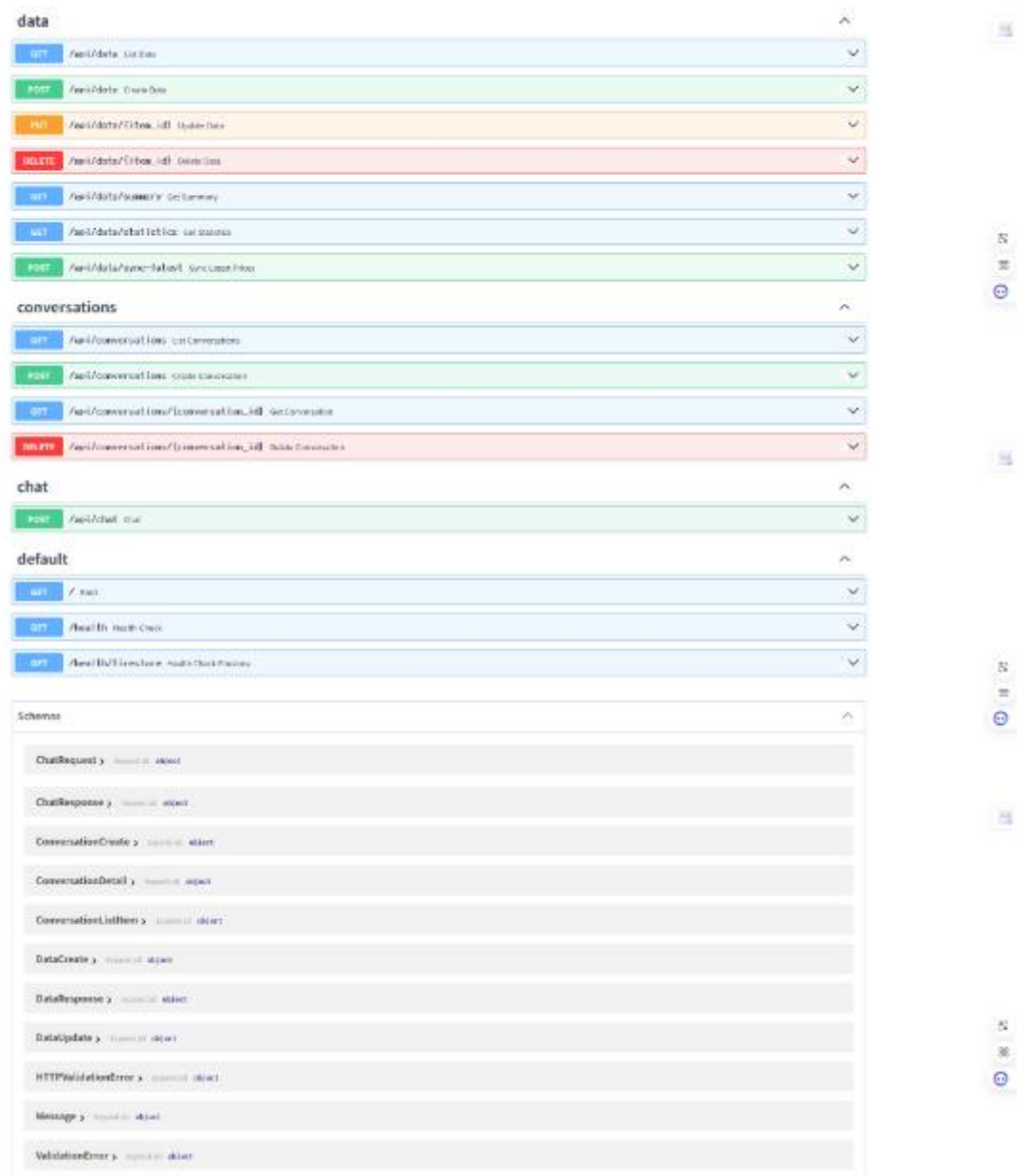


그림 4. 배포된 백엔드(Render)의 Swagger UI — data / conversations / chat 태그별 전체 엔드포인트 확인

7. 보너스 과제 구현 내역

7-1. Function Calling + 멀티채널 연동(MCP Server)

AI가 필요하다고 판단할 경우 내부 도구(get_data_summary, get_conversation_history)를 스스로 호출하도록 chat_with_tools()를 구현하였다. 예: "최신 데이터 요약을 다시 보여줘" → AI가 get_data_summary를 스스로 호출해 실시간 Firestore 데이터를 반영한 답변을 생성. 외부 채널 연동은 MCP(Model Context Protocol) Server로 구현하였으며, 이미 만든 FastAPI 엔드포인트를 그대로 감싸는 방식으로 로직 중복 없이 설계하였다. MCP Inspector로 3개 도구(get_data_summary, get_conversation_history, ask_ai_assistant)의 정상 호출을 검증하였다.

호출 흐름:
사용자 질문
→ AI가 도구 호출 필요 여부 판단
→ (필요시) get_data_summary / get_conversation_history 실행
→ 실행 결과를 AI에게 재전달
→ AI가 최종 자연어 답변 생성

7-2. 인사이트·UX 고도화

항목	구현 내용
통계 확장	GET /api/data/statistics — 표준편차, 변동성(%), 최근 7일 변화율 제공
시각화	Canvas API로 직접 구현한 금/은 시세 추이 라인 차트
내보내기	CSV / JSON 다운로드 버튼
다크 모드	CSS 변수 기반 토글, localStorage 저장으로 재방문 시 유지
추가 구현	USD/KRW 통화 전환(실시간 환율 API, 실패 시 대체 환율 폴백), 기간별 조회 필터, 야후 파이낸스 최신 시세 수동 동기화(sync-latest, 중복 방지 로직 포함)

8. 운영·보안 보완 사항

8-1. 컨텍스트 주입의 한계와 완화 방안

본 서비스의 핵심 메커니즘인 시스템 프롬프트 데이터 주입 방식에는 아래와 같은 구조적 한계가 있으며, 이를 인지하고 각각 완화 조치를 적용하였다.

한계	완화 방안
데이터 누적 시 프롬프트 비대화 우려	원본 레코드가 아닌 계산된 통계 요약만 주입 — 레코드 수와 무관하게 요약 크기가 거의 고정됨
요약 시점과 실제 조회 시점의 시차	매 /api/chat 요청마다 요약을 캐시 없이 새로 계산. 실시간성이 중요한 경우 use_tools=true로 AI가 직접 재조회하도록 유도
시스템 프롬프트가 외부 AI 서버로 전송됨	개인 식별 정보가 아닌 시세 데이터(날짜/가격/구분)만 다루어 프라이버시 리스크가 낮음. 민감정보 마스킹 로직은 향후 과제로 남김
요약 dict를 문자열 포매팅에 직접 사용	각 필드에 기본값("정보 없음" 등)을 제공하여 빈 데이터에도 예외 없이 동작하도록 처리
요약에 없는 세부 질문에 추측 답변 가능성	Function Calling으로 AI가 필요 시 최신 데이터를 직접 조회하도록 하여 추측 답변 가능성을 낮춤

8-2. 서비스 계층 책임(Service Layer Contracts)

함수	책임	입력 / 출력
get_firestore_client()	Firestore 클라이언트 싱글톤 초기화/반환	없음 / client 객체
calculate_summary(records)	기간/개수/평균·최대·최소/추세 / 금은비율 계산	list[dict] / {period, count, metrics, trend, ratio}

함수	책임	입력 / 출력
calculate_statistics(records)	표준편차, 변동성(%), 최근 7일 변화율 계산	list[dict] / memo별 통계 dict
get_ai_provider()	AI_PROVIDER 값에 따라 Gemini/OpenAI 반환	없음 / AIProvider 구현체
AIProvider.chat() / chat_with_tools()	시스템 프롬프트+기록+메시지로 AI 응답 생성	str, str, list[dict] / str

8-3. 데이터 검증 규칙

서버(Pydantic)는 date/value/memo를 타입 수준에서 필수값으로 강제하며, memo는 자유 문자열("금"/"은"으로 제한하지 않음)이다.

클라이언트는 <input type="date">와 <select>로 형식·값을 UI 레벨에서 제한한다. 즉 프론트 UI를 우회한 직접 API 호출 시 "금"/"은" 외의 값이 들어갈 수 있는 점은 알려진 제한사항으로 남겨두었다.

8-4. 저장 실패 시 처리 정책

데이터/대화 저장(POST) 실패 시 alert()로 즉시 사용자에게 알리며, 폼 입력값을 초기화하지 않아 재시도가 가능하다. 자동 재시도(retry)는 구현하지 않았다.

Firestore의 단일 문서 set()/update()는 그 자체로 원자적이거나, "저장 성공 + summary 계산" 같은 여러 단계를 하나의 트랜잭션으로 묶지는 않았다.

동시성(여러 클라이언트의 동시 수정)은 Firestore의 last-write-wins 동작을 그대로 따른다.

8-5. 기간 필터 우선순위 및 동기화

프론트엔드 화면(요약 카드/차트/테이블)의 기본값은 항상 Firestore 전체 기간 데이터를 기준으로 하며, 클라이언트의 기간 필터(시작일~종료일)는 차트·테이블의 화면 표시 범위만 조정한다.

2차 보완으로 GET /api/data/summary, GET /api/data/statistics에 선택적 쿼리 파라미터 start/end(YYYY-MM-DD)를 추가하여 서버 측에서도 특정 기간만 집계할 수 있도록 확장하였다.

다만 POST /api/chat(AI 컨텍스트 주입)은 아직 이 파라미터와 연동하지 않아 항상 전체 기간 기준으로 답변하며, 이는 알려진 제한사항으로 남긴다.

8-6. 채팅 입력 서버측 검증(2차 보완)

평가 피드백을 반영하여 POST /api/chat의 Pydantic 스키마(ChatRequest)에 서버측 검증을 추가하였다.

message는 1~2000자로 제한하고 제어문자 제거 후 공백만 남으면 거부하며, conversation_id는 Firestore 문서 ID 조회에 그대로 사용되므로 [A-Za-z0-9_-]+ 정규식 형식만 허용한다.

2000자 초과 메시지, 공백만 있는 메시지, 경로 순회 형태(./etc/passwd)의 conversation_id 세 가지 모두 422로 정상 차단되는 것을 배포 환경에서 직접 검증하였다.

아울러 시스템 프롬프트와 사용자 메시지가 API 레벨에서 서로 다른 필드로 분리 전달되고, Function Calling으로 AI가 실행 가능한 동작이 get_data_summary/get_conversation_history 두 가지로 고정되어 있어, 프롬프트 인젝션이 일부 성공하더라도 실질적 피해 범위가 구조적으로 제한된다.

8-7. 배포 및 운영 참고사항

배포 상태는 Render 대시보드(dashboard.render.com)에서 별도 확인 가능하다.

콜드 스타트(무료 티어, 최대 약 1분)에 대해서는 별도 워밍 스케줄러 없이 안내 문구로 대응한다.

ALLOWED_ORIGINS는 로컬 개발 주소와 실제 배포 도메인만 등록하며 와일드카드는 사용하지 않는다.

비밀 관리는 Render/Vercel의 환경변수 기능을 그대로 사용하며, 별도의 비밀 관리 서비스는 도입하지 않았다.

8-8 알려진 제한사항

- ① 모바일 반응형은 iPhone 실기기 캡처로 검증하였으나, 스크린리더 등 정식 접근성(a11y) 감사 도구 검증은 진행하지 않았다.
- ② 네트워크 오류 시 자동 재시도 로직이 없다.
- ③ Firestore 쿼리에 별도 복합 인덱스를 설계하지 않았다(현재 쿼리 패턴이 단순하여 자동 인덱스로 충분).
- ④ memo 필드는 API 레벨에서 값이 제한되어 있지 않다.

9. 트러블슈팅 기록

문제	원인	해결
Firestore 403 오류 (SERVICE_DISABLED)	Firestore Database 생성이 실제로는 완료되지 않은 상태였음	Firebase 콘솔에서 Database를 다시 정상 생성
uvicorn 명령 인식 실패	다른 프로젝트의 venv가 활성화된 상태로 명령 실행	터미널마다 활성화된 venv 경로(Get-Command uvicorn/python)를 확인
Render 배포 실패(pywin32)	Windows 로컬 requirements.txt에 Windows 전용 패키지가 포함되어 Linux 배포 환경에서 설치 실패	requirements.txt에 pywin32==312; sys_platform == 'win32' 형태로 조건부 설치 명시
MCP 서버 ModuleNotFoundError	MCP Python SDK 2.0에서 mcp.server.fastmcp 모듈이 제거되고 MCPServer로 변경됨	커뮤니티 표준 독립 패키지 fastmcp(PrefectHQ)로 전환
8000번 포트 재사용 불가	이전 uvicorn 프로세스가 완전히 종료되지 않고 남아있음	netstat -ano로 점유 PID 확인 후 taskkill로 강제 종료
배포 사이트 좌측 상단에 의도치 않은 텍스트 노출	<head> 태그 안에 <th> 태그가 실수로 삽입되어 브라우저가 <body> 시작 지점으로 밀어냄	해당 코드를 올바른 위치(<thead> 안)로 이동
POST /api/data/sync-latest 405 오류	배포된 프론트엔드가 이미 Render 백엔드를 바라보고 있었으나, 새로 추가한 엔드포인트 코드가 아직 배포에 반영되지 않음	변경된 백엔드 코드를 커밋/푸시하여 Render 재배포

10. 최종 결론 및 학습 성과

본 미션을 통해 "시계열 데이터 분석 → DB 저장 → AI 컨텍스트 주입"이라는 전체 파이프라인을 처음부터 끝까지 직접 구현하였다. 특히 AI 프로바이더를 전략 패턴으로 추상화하여 Gemini/OpenAI를 환경변수만으로 전환 가능하도록 설계한 경험은, 특정 벤더에 종속되지 않는 서비스 아키텍처 설계 원칙을 체득하는 계기가 되었다.

Firestore 연동 과정에서 겪은 서비스 활성화 문제, Windows-Linux 배포 환경 차이로 인한 패키지 오류, MCP SDK의 빠른 버전 변화 대응 등은 실무에서도 흔히 마주치는 상황이며, 각 문제를 로그 기반으로 정확히 진단하고 해결하는 과정에서 트러블슈팅 역량을 키울 수 있었다.

특히 자체 평가 및 외부 검토 과정을 통해, 컨텍스트 주입 방식이 갖는 구조적 한계(토큰 비대화, 시차, 프라이버시, 포맷 일관성, 추측 답변 가능성)를 명시적으로 문서화하고 각각의 완화 방안을 정리한 것은, 단순히 기능을 구현하는 것을 넘어 그 설계의 트레이드오프를 스스로 점검하는 습관을 기르는 데 도움이 되었다. 저장 실패 정책, 기간 필터 우선순위, 서비스 계층 책임 등을 명문화하는 과정에서 코드뿐 아니라 운영 관점의 문서화 역량도 함께 향상시킬 수 있었다.

보너스 과제로 구현한 MCP Server 연동은 하나의 FastAPI 백엔드를 여러 채널(웹 프론트엔드, MCP 클라이언트)에서 재사용 가능한 형태로 설계하는 경험을 제공했으며, Function Calling을 통해 AI가 스스로 판단하여 내부 도구를 호출하는 흐름을 직접 구현하고 검증할 수 있었다.

11. 요구사항 전체 이행 요약표

구분	항목	상태
필수	데이터 기반 AI 채팅 (컨텍스트 주입)	완료
필수	데이터 관리 CRUD	완료
필수	대화 기록 저장 및 불러오기	완료
필수	백엔드 배포(Render) + Swagger	완료
필수	프론트엔드 배포(Vercel) + README	완료
필수	.env / .gitignore / .env.example 보안 구성	완료
보너스 A	Function Calling + MCP Server 외부 채널 연동	완료
보너스 B	통계 확장 / 시각화 / 내보내기 / 다크모드	완료
운영 보완	컨텍스트 주입 한계 문서화, 서비스 계약, 검증 규칙, 저장 실패 정책	완료
운영 보완	Swagger·모바일·데스크톱 실증 스크린샷 확보	완료
운영 보완	채팅 입력 서버측 검증(길이/공백/ID 형식), 요약·통계 API 기간 필터	완료
추가	USD/KRW 통화 전환, 기간 필터, 야후 파이낸스 동기화	완료